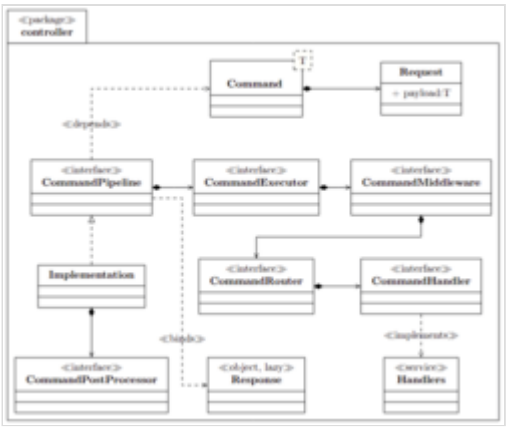
# Command Query Responsibility Segregation

In **modern enterprise systems**, increasing architectural complexity has introduced significant challenges related to scalability, maintainability, and runtime performance. Implementations of **Command Query Responsibility Segregation (CQRS)** and event-driven designs frequently integrate core business logic with cross-cutting concerns such as logging, retry policies, and audit propagation.[2] This combination can lead to tightly coupled components, reducing modularity and complicating long-term evolution.



Class Diagram of Command Query Responsibility Segregation[1]

Modern CQRS frameworks aim to decouple these operational aspects from domain logic to improve system adaptability. Drawing on established design principles—such as the Chain of Responsibility and Strategy patterns[3]—alongside declarative programming paradigms, CQRS models command processing as a functional pipeline[4] $P$. Each processing stage $s \in S$ (for example, routing, middleware execution, handler invocation, or post-processing) is coordinated through dynamically extensible interceptor chains $I$.

This architectural[5] approach is intended to provide predictable latency, high throughput,[6] strong type safety,[7] and reliable idempotency characteristics, making it suitable for resilient large-scale distributed applications. In information technology, **Command Query Responsibility Segregation** (CQRS) is a software architecture that extends the idea behind command–query separation (CQS) to the level of services.[8][9] Such a system will have separate interfaces to send queries and to send commands. As in CQS, fulfilling a query request will only retrieve data and will not modify the state of the system (with some exceptions like logging access), while fulfilling a command request will modify the state of the system.

Many systems push the segregation to the data models used by the system. The models used to process queries are usually called *read models* and the models used to process commands *write models*.

Although its origin is usually attributed to Greg Young in 2010,[8] everything indicates that the precursor of CQRS was Udi Dahan who in August 2008 published on his blog a training course that aimed to apply CQRS together with SOA [10] and in more detail in December 2009 in the article Clarified CQRS.[11]

## References

1. Martin, Robert C. (2003). *UML for Java Programmers*. Prentice Hall PTR. ISBN 9780131428485.
2. Marin, Marius; Deursen, Arie Van; Moonen, Leon (2007). "Identifying crosscutting concerns using fan-in analysis" (https://dl.acm.org/doi/abs/10.1145/1314493.1314496). *ACM Transactions on Software Engineering and Methodology*. **17** – via ACM New York, NY, USA.

3. Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John (1994). *Design patterns, software engineering, object-oriented programming* (1st ed.). Addison-Wesley Professional Computing Series. ISBN 9780321700698.
4. Ramamoorthy, Chittoor V; Li, Hon Fung (1977). "Pipeline architecture" (https://dl.acm.org/doi/abs/10.1145/356683.356687). *ACM Computing Surveys*. **9**: 61–102 – via ACM New York, NY, USA.
5. Richards, Mark; Ford, Neal (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media. ISBN 9781492043423.
6. Thompson, Martin; Farley, Dave; Barker, Michael; Gee, Patricia; Stewart, Andrew (2011). "Disruptor: High performance alternative to bounded queues for" (https://lmax-exchange.github.io/disruptor/files/Disruptor-1.0.pdf) (PDF). *Github.io*.
7. Leinberger, Martin (2021). *Type-safe programming for the semantic web*. IOS Press. ISBN 9781643681979.
8. Young, Greg. "CQRS Documents" (https://cqrs.wordpress.com/wp-content/uploads/2010/11/cqrs_documents.pdf) (PDF). Retrieved 2024-10-09.
9. Fowler, Martin. "CQRS" (http://martinfowler.com/bliki/CQRS.html). Retrieved 2011-07-14.
10. Dahan, Udi. "Advanced Distributed Systems Design using SOA & DDD" (https://udidahan.com/training/). Retrieved 2024-10-09.
11. Dahan, Udi. "Clarified CQRS" (https://udidahan.com/2009/12/09/clarified-cqrs/). Retrieved 2024-10-09.

## External links

- CQRS Journey by Microsoft patterns & practices (https://learn.microsoft.com/en-us/previous-versions/msp-n-p/jj554200(v=pandp.10))
- DDD/CQRS/Event Sourcing List (https://groups.google.com/group/dddcqrs)
- The CQRS Frequently Asked Questions (http://www.cqrs.nu)
- CQRS - a new architecture precept based on segregation of commands and queries (http://www.h-online.com/developer/features/CQRS-an-architecture-precept-based-on-segregation-of-commands-and-queries-1803276.html)
- Rethink How You Build Software (https://www.cqrs.com/)
- CQRS HelloWorld Implementation by Kapil Panchal (https://github.com/NewCommandProcessingInfrastructure/CQRS)